AI 對話紀錄 — 物聯網應用與資料分析 HW2

CRISP-DM 線性回歸互動網頁 (Kaggle Playground S4E5 洪水預測) 的開發過程 ｜ AI 工具：Claude Code ｜ 10/17 17:53 ~ 10/17 22:46 (UTC+8) ｜ 使用者訊息 32 則

說明：完整保留使用者訊息、貼上的截圖與 AI 的文字回覆；AI 執行的工具動作 (執行指令、讀寫檔案、瀏覽器操作) 以「🔧」一行摘要呈現，工具的原始輸出與內部推理過程略過。

👤 使用者 10/17 17:53

data 裡面的資料來源是https://www.kaggle.com/datasets/brahimenesulusoy/istanbul-apartment-prices-2026/data
請將程式改成以這個資料來分析的內容，功能不要變

🔧 **Bash** — $ find . -not -path '*/.*' -not -path '*/node_modules*' -not -path '*venv*' | head -50; ls -la data; for f in da …
🔧 **Bash** — $ cat requirements.txt; wc -l main.py index.html; cat main.py
🔧 **Read** — hw2/main.py
🔧 **Bash** — $ cat index.html; grep -n -i "startup\|boston\|problem\|PROBLEMS\|key ==\|\"key\"\|medv\|profit\|R&D\|State" mai …
🔧 **Read** — hw2/main.py
🔧 **Read** — hw2/main.py

🤖 AI 助理 (Claude Code) 10/17 17:54

I'm reading through `main.py` (the existing 50 Startups / Boston app) to see what needs to change. Next I'll check the Istanbul CSV's columns and distributions.

🔧 **Bash** — $ python3 -I -c " …
🔧 **Bash** — $ ls ~/miniconda3/bin/python ~/anaconda3/bin/python /opt/homebrew/bin/python3* 2>/dev/null; which -a python pyth …
🔧 **Bash** — $ for p in /usr/local/bin/python /opt/homebrew/bin/python3.12; do echo $p; $p -c "import pandas,streamlit,sklear …
🔧 **Bash** — $ S=scratchpad; /opt/homebrew/bin/python3.12 -m venv $S/venv && $S/venv/bin/pip install -q streamlit pandas nump …
🔧 **Bash** — $ scratchpad/venv/bin/python -I -c " …

🤖 AI 助理 (Claude Code) 10/17 17:56

Now I'll make the changes. The plan: point both built-in problems (total price / price per m²) at the Istanbul CSV, add a cleaning and sampling step, and replace the Startups/Boston-specific text.

🔧 **Bash** — $ cp main.py scratchpad/main_orig.py; scratchpad/venv/bin/python -I -c " …

🤖 AI 助理 (Claude Code) 10/17 17:57

Now editing the constants and loader.

🔧 **Edit** — hw2/main.py
🔧 **Edit** — hw2/main.py

🤖 AI 助理 (Claude Code) 10/17 17:57

Next, the sidebar, data loading and default features.

🔧 **Bash** — $ python3 - <<'EOF' ...

🤖 **AI 助理 (Claude Code)** 10/17 17:57

Now the CRISP-DM text sections and the Startups-specific bits.

🔧 **Bash** — $ python3 - <<'EOF' ...

🤖 **AI 助理 (Claude Code)** 10/17 17:58

Updating `index.html` (the stlite browser version) to load the new CSV:

🔧 **Bash** — $ python3 - <<'EOF' ...
🔧 **Bash** — $ S=scratchpad; sed -i '' 's#from_file("main.py"#from_file("hw2/main.py"#' $S/t.py; cd "hw2" && $S/venv/bin/pyth ...
🔧 **Bash** — $ S=scratchpad; cd "hw2" && $S/venv/bin/python $S/t.py 2>/dev/null | grep -E "^[01] [012] "

🤖 **AI 助理 (Claude Code)** 10/17 18:00

All six combinations (2 problems × 3 models) run without errors. Next I'll check the model results and a few edge paths: adding district, full One-Hot, and auto-adopting the best model.

🔧 **Bash** — $ S=scratchpad; cat > $S/t2.py <<'EOF' ...

🤖 **AI 助理 (Claude Code)** 10/17 18:06

Everything works. Adding `district` (68 encoded features) makes the model search take about 5 minutes, so I'll add a note about that in the sidebar.

🔧 **Edit** — hw2/main.py
🔧 **Bash** — $ S=scratchpad; cd "hw2" && $S/venv/bin/python $S/t.py 2>/dev/null | grep -E "^[01] [012] "

📝 **本輪摘要**：main.py and index.html now use the Istanbul apartment data. The two dropdown problems are total price and price per m², and all features still work. All six problem × model combinations ran without errors; I didn't open the browser (stlite) version.

🤖 **AI 助理 (Claude Code)** 10/17 18:07

I've switched the program to analyze the Istanbul data in `data/`, and all features work as before. I ran 6 combinations (2 problems × 3 models) and none of them errored. I only tested this on my machine, not the browser version (`index.html`).

What changed:

- **The two dropdown problems** are now from this dataset:
- 📊 Predict total apartment price (`price`)
- 📐 Predict price per square meter (`price_per_sqm`)
- Each problem leaves the other price column out of the default features. One is computed from the other, so including it would leak the answer to the model.

- **Data loading:** Kaggle requires a login to download, so the program reads `data/istanbul_apartment_prices_2026.csv` directly. The source is shown as the Kaggle link.
- **Data cleaning** (listed in Stage 3, "Data Preparation"):
- Dropped ID, text and date columns; columns over 55% missing; and `total_rooms`, which is just `rooms + halls`.
- Removed values above the 99th or below the 1st percentile of `price` and `price_per_sqm`, plus unreasonable values (e.g. `net_sqm` outside 30–400 m^2). About 23,000 rows remain.
- Added a new column `side` marking the European or Asian side of Istanbul. It replaces `district` (38 districts) in the default features.
- Merged the rare categories of `heating_type` and `deed_status` into Other.
- **New sidebar setting, "內建資料取樣筆數" (sample size):** it uses a random 2,000 rows by default. With the full data, feature selection and the nested cross-validation are too slow to use interactively.
- **Text updated:** the business scenario, questions and One-Hot explanations are rewritten for Istanbul apartments. I removed the "reference page feature order" checkbox, which only worked with 50 Startups.
- **`index.html`:** now loads the new CSV.

Results and speed:

Problem	Features recommended by the 1-SE rule	Test R^2
Total price	4	0.45
Price per m^2	8	0.27

R^2 is modest because a linear model on raw prices doesn't fit very well. On my machine the default multiple linear regression takes about 20 seconds; simple regression and AR take under 2 seconds. If you add `district` back, the encoded features grow to 68 and the best-model search takes about 5 minutes. The sidebar now says this.

One thing to know: in Stage 6, "Deployment", each category input defaults to its alphabetically first option (e.g. Asian, Furnished). That's how the original program worked too, which is why the default predicted price comes out on the high side.

👤 **使用者** 10/17 18:11

如果我不要用網頁載入會跑很久的套件，要怎麼改

🔧 **Bash** — $ grep -n "trendline\|statsmodels\|stats\.\|from scipy\|^from sklearn\|^import" main.py

🤖 **AI 助理 (Claude Code)** 10/17 18:11

`statsmodels` is used in only one place: the `trendline="ols"` scatter plot. I'll replace it with a numpy fit.

🔧 **Read** — hw2/main.py
🔧 **Edit** — hw2/main.py
🔧 **Bash** — $ sed -i '' '/^statsmodels$/d' requirements.txt; sed -i '' 's/requirements: \["scikit-learn", "scipy", "plotly", ...

🤖 **AI 助理 (Claude Code)** 10/17 18:12

Now I'll uninstall statsmodels from the test environment and rerun the tests, to confirm the app really runs without it.

🔧 **Bash** — $ S=scratchpad; $S/venv/bin/pip uninstall -y -q statsmodels patsy; cd "hw2" && $S/venv/bin/python $S/t.py 2>/dev ...

📝 **本輪摘要**：I removed statsmodels: the scatter plot's trend line is now computed with numpy, and statsmodels is gone from requirements.txt and index.html. All 6 tests pass with statsmodels uninstalled. I didn't measure the load-time difference in the browser.

🤖 **AI 助理 (Claude Code)** 10/17 18:13

I removed **statsmodels**, the one package that could be dropped without losing any feature. I haven't measured how much faster the web page now loads.

What changed:

- **`main.py`:** statsmodels was only used for the trend line (`trendline="ols"`) on the "特徵與目標變數散佈圖" (feature vs. target scatter plot). That line is now fit with `np.polyfit`, still one line per category, and the legend shows the equation.
- **`requirements.txt` and `index.html`:** statsmodels is gone from both, so the page no longer downloads it (or `patsy`, which comes with it).
- **Tests:** I uninstalled statsmodels from the test environment and reran all 6 combinations (2 problems × 3 models). All of them run without errors.

What's still slow to load: scikit-learn + scipy. These are the two biggest packages in the browser, and dropping them takes much more work:

- scikit-learn needs scipy, so you can only drop both together.
- scipy is also used directly, for the t and F distributions (p-values, confidence intervals, the F test). Those could be computed with numpy.
- scikit-learn covers nearly all the modeling: LinearRegression, cross-validation, RFE / RFECV / SFS, LassoCV / ElasticNetCV, mutual information and random forest. Linear regression, cross-validation, RFE/SFS and Lasso can be rewritten with numpy; the main cost is a lot of code.
- **Random forest and mutual information are the hard part.** Pure-Python versions would run very slowly in the browser. They'd have to be removed or replaced with simpler methods, which would change the program's features.

So my recommendation is to stop here for now. If startup is still too slow, the next easy speedup has nothing to do with packages: replace the 5.4 MB CSV with a pre-cleaned version that keeps only the columns the app uses. That cuts download and parsing time without touching any features. I can do that next, or do the full rewrite that drops scikit-learn and scipy.

👤 **使用者** 10/17 18:19

1. RMSE圖幫我把最低的星星預設顯示
2. Feature Selection 的部分要自己篩選變數的話，最上面的篩選方式可不可以合併到這邊

🔧 **Bash** — $ grep -n "ctrl_box\|head_box\|tabs = st.tabs\|with tabs\[2\]\|側邊欄\|legendonly\|特徵篩選過程\|步驟①" main.py
🔧 **Read** — hw2/main.py

🤖 **AI 助理 (Claude Code)** 10/17 18:19

Now making the edits: the star, the containers, and the text that points to the old location.

🔧 **Bash** — $ python3 - <<'EOF' …
🔧 **Bash** — $ S=scratchpad; cat > $S/t3.py <<'EOF' …

👤 **使用者 (工作中補充)** 10/17 18:21

另外是幫我把它改成一頁式網頁，不要用tab切換

🔧 **Bash** — $ grep -n "tabs\[\|st.header(\|分頁\|st.tabs" main.py
🔧 **Read** — hw2/main.py
🔧 **Bash** — $ python3 - <<'EOF' …
🔧 **Bash** — $ python3 - <<'EOF' …

🤖 **AI 助理 (Claude Code)** 10/17 18:23

The automated tests pass. Now I'll start the app and look at it in the browser to check the page order and layout.

🔧 **Bash** — $ scratchpad/venv/bin/streamlit run main.py --server.port 8765 --server.headless true > scratchpad/st.log 2>&1
🔧 **navigate** — http://localhost:8765
🔧 **browser_batch** — 瀏覽器操作 (navigate, computer, computer)
🔧 **browser_batch** — 瀏覽器操作 (computer, javascript_tool)

🤖 **AI 助理 (Claude Code)** 10/17 18:24

The page order is correct. Next I'll check the RMSE chart's star and the top section.

🔧 **find** — 特徵數量 vs 模型表現
🔧 **browser_batch** — 瀏覽器操作 (computer, computer, computer)

🤖 **AI 助理 (Claude Code)** 10/17 18:24

The star now shows by default. Last, the jump links at the top:

🔧 **browser_batch** — 瀏覽器操作 (find, javascript_tool, computer)

🤖 **AI 助理 (Claude Code)** 10/17 18:24

Everything checks out. Stopping the test server:

🔧 **TaskStop** —

📝 **本輪摘要**：All three changes are done: the lowest-RMSE star shows by default, the feature-selection controls are inside stage 3's "🔎 特徵篩選過程" section, and the app is now a single page with jump links. The automated tests and the in-browser check both pass. The One-Hot encoding setting is still above

stage 1.
❓ **Should the One-Hot setting also move into stage 3?**

👤 **使用者** 10/17 19:30

預設這個方式



🔧 **Edit** — hw2/main.py
🔧 **Bash** — $ S=scratchpad; cat > $S/t4.py <<'EOF' …

📝 **本輪摘要**：Feature selection now defaults to "Embedded 嵌入法 → LassoCV (L1 正則化)". With the default data it keeps 23 of the 24 features. All tests pass.

🤖 **AI 助理 (Claude Code)** 10/17 19:32

The page now opens with 篩選類別 (filter category) set to **Embedded 嵌入法** and 篩選方法 (method) set to **LassoCV (L1 正則化)**, as in your screenshot. You can still switch to any other method.

With the default data (2,000-row sample, seed 0), Lasso keeps 23 of the 24 features. The defaults are confirmed, and all the earlier tests still pass.

The change is in the ① 選擇特徵篩選演算法 (choose feature-selection algorithm) part of `main.py`.

👤 **使用者** 10/17 20:51

好像沒有回答最一開始設定的問題欸

🎯 商業場景設定

您是伊斯坦堡的房地產估價顧問，面對的挑戰是：**「如何依據公寓的面積、房間數、屋齡、樓層、所在區域 (歐洲岸 / 亞洲岸) 等條件，合理估計一間公寓的掛牌總價 (土耳其里拉 TL)？」**

需要回答的問題：

- 淨面積 (net_sqm)、房間數、衛浴數與總價之間是否存在線性關係？
- 每多 1 m² 淨面積，總價平均增加多少 TL？
- 位於歐洲岸或亞洲岸、屋齡、是否在社區 (is_in_complex) 內，對總價是否有顯著影響？

🤖 **AI 助理 (Claude Code)** 10/17 20:52

I'll add a section that answers the stage 1 questions from the actual analysis results. First I'll check how the questions are defined in the code.

🔧 **Bash** — $ grep -n "if is_custom:\|for q in qs\|# ---------- 6\|^def vif_table\|^# ----.*\n# 特徵篩選\|coef_cmp = " mai …

🤖 **AI 助理 (Claude Code)** 10/17 20:52

I'll add the helper functions, the "回答商業問題" (answer the business questions) section at the end of stage 5, and a link in stage 1.

🔧 **Bash** — $ python3 - <<'EOF' …
🔧 **Bash** — $ S=scratchpad; cat > $S/t5.py <<'EOF' …
🔧 **Bash** — $ S=scratchpad; cd "hw2"; grep -n "b_ols\b.*=\|w_ols\b.*=" main.py | grep -v "==" | head; grep -n "num_cols = \[…

👤 **使用者 (工作中補充)** 10/17 20:54

最後是把ui做漂亮一點，幫我設計好看ui

🤖 **AI 助理 (Claude Code)** 10/17 20:55

I'll finish these two fixes first, then work on the UI design.

🔧 **Bash** — $ S=scratchpad; cd "hw2"; $S/venv/bin/python -I - <<'EOF' 2>/dev/null …
🔧 **Bash** — $ python3 - <<'EOF' …

🤖 **AI 助理 (Claude Code)** 10/17 20:56

The AR results are correct now. Next I'll check the `price_per_sqm` answers and tweak the AR trend wording.

🔧 **Bash** — $ python3 - <<'EOF' …

🤖 **AI 助理 (Claude Code)** 10/17 20:57

A trailing comma turned one answer into a tuple; I'll fix that. Then I'll start on the UI design, first loading the chart-design guidance so the chart colors match the new design.

🔧 **Bash** — $ python3 - <<'EOF' …
🔧 **Skill** —
🔧 **Bash** — $ cat /private/tmp/claude-501/bundled-skills/2.1.299/7f65d9d268b7da26076dc3caebfdd972/dataviz/references/palette …
🔧 **Bash** — $ sed -n '/## Diverging pair/,/## Texture/p' /private/tmp/claude-501/bundled-skills/2.1.299/7f65d9d268b7da26076d …

🤖 **AI 助理 (Claude Code)** 10/17 20:58

I'll use the skill's validated default palette (blue, orange, aqua…) unchanged, so it doesn't need re-validating. Next I'll check how the charts are called in the code.

🔧 **Bash** — $ grep -c "plotly_chart(" main.py; grep -n "plotly_chart(" main.py | grep -v "use_container_width=True)" ; grep …

🤖 **AI 助理 (Claude Code)** 10/17 20:58

Now the main UI changes: theme, CSS, hero banner, section header cards, Q&A cards, and unified chart colors.

🔧 **Bash** — $ grep -n "^st.set_page_config\|sec.divider()\|^with head_box" -A4 main.py | head -30; grep -n "st.header(" main …
🔧 **Bash** — $ python3 - <<'EOF' …
🔧 **Bash** — $ grep -c 'line_color="red"' main.py; grep -c 'html\|PALETTE' main.py

🤖 **AI 助理 (Claude Code)** 10/17 20:59

The script stopped before writing anything, so the file is unchanged. I'll save the script to a file, fix the count, and rerun.

🔧 **Write** — scratchpad/ui_patch.py
🔧 **Bash** — $ python3 scratchpad/ui_patch.py main.py && mkdir -p .streamlit && cat > .streamlit/config.toml <<'EOF' …

🤖 **AI 助理 (Claude Code)** 10/17 21:00

Patch applied. Now the tests:

🔧 **Bash** — $ S=scratchpad; cd "hw2" && $S/venv/bin/python $S/t.py 2>/dev/null | grep -E "^[01] [012] "; $S/venv/bin/python …

🤖 **AI 助理 (Claude Code)** 10/17 21:01

All tests pass. Next I'll open the browser to check the visual result (the streamlit server needs to run from the project folder so it picks up `.streamlit/config.toml`).

🔧 **Bash** — $ scratchpad/venv/bin/streamlit run main.py --server.port 8765 --server.headless true >

scratchpad/st.log 2>&1
🔧 **browser_batch** — 瀏覽器操作 (computer, navigate, computer, computer)
🔧 **browser_batch** — 瀏覽器操作 (navigate, computer, computer, computer)
🔧 **Bash** — $ grep -n "ctrl_box\|oh_choice\|bcols" main.py
🔧 **Bash** — $ python3 - <<'EOF' …
🔧 **browser_batch** — 瀏覽器操作 (navigate, computer, computer, computer)

🤖 **AI 助理 (Claude Code)** 10/17 21:02

That's much cleaner. I'll change the last KPI card to just the test-set row count (training count goes in the tooltip), then check the rest at a wider width.

🔧 **Bash** — $ python3 - <<'EOF' …
🔧 **resize_window** —
🔧 **browser_batch** — 瀏覽器操作 (navigate, computer, computer, computer)
🔧 **computer** — screenshot

🤖 **AI 助理 (Claude Code)** 10/17 21:03

Looks good. Next I'll click the "🎯 商業問題解答" (business answers) link at the top to check the jump links and the answer cards.

🔧 **browser_batch** — 瀏覽器操作 (computer, computer, computer)

🤖 **AI 助理 (Claude Code)** 10/17 21:03

The jump link and answer cards both work. Last, a check on the chart colors (correlation heatmap and the RMSE chart):

🔧 **find** — 相關係數熱圖
🔧 **browser_batch** — 瀏覽器操作 (computer, computer, find)
🔧 **browser_batch** — 瀏覽器操作 (javascript_tool, computer, computer, javascript_tool)

🤖 **AI 助理 (Claude Code)** 10/17 21:03

Charts are using the new colors. One small fix: the single-series histogram (price distribution) doesn't need a legend, so I'll hide it.

🔧 **Bash** — $ python3 - <<'EOF' …
🔧 **TaskStop** —
🔧 **resize_window** —

📝 **本輪摘要**：Added a "🎯 回答商業問題" section that answers the stage 1 questions from the actual results, and redesigned the UI. Along the way I fixed an OLS precision bug and a target leak in simple regression. All 6 tests pass and I checked the page in the browser; only index.html (the stlite browser version) hasn't been opened.

🤖 **AI 助理 (Claude Code)** 10/17 21:04

Both requests are done. The questions from stage 1 are now answered in a new section, and the interface has been redesigned. All 6 combinations (2 problems × 3 models) run without errors, and I checked the result in a browser. I haven't opened the browser version (`index.html`).

1. Answering the stage 1 questions

There's a new "🎯 回答商業問題" (answer business questions) section at the end of stage 5. Each question gets a card, and the answers are generated automatically from the analysis results. Stage 1 and the top of the page both link to it. Some findings with the default data (2,000-row sample):

- **Total price, Q2:** each extra 1 m^2 of net area adds about **111,350 TL** to the price (95% CI 103,214–119,486).
- **Total price, Q3:** holding other conditions equal, the European side averages about 1.76M TL *less* than the Asian side, and being in a complex (`is_in_complex`) about 1.42M TL less. Both are significant. Building age is not significant.
- **Price per m^2, Q3:** larger apartments do *not* have a lower price per m^2. The correlation is r = 0.046, slightly positive and very weak, so there's no economy-of-scale effect here.

For simple regression and AR, the answers come from that model's slope, autocorrelation, forecast and so on. If you change the target variable, the questions switch to generic ones.

2. UI design

- **Header banner:** blue-purple gradient with the dataset, target, model and sample size, plus buttons that jump to each stage.
- **Result cards:** a row of cards right below the banner for test R^2, RMSE, MAE and test-set size. Large numbers are shortened (e.g. 6.72M); hover for the exact value.
- **Stage headers:** each stage's title is a card with a numbered badge.
- **Cards:** charts, tables and metrics have rounded white card backgrounds, and the font is Noto Sans TC.
- **Sidebar:** gets its own header block. The One-Hot setting moved into a collapsible card.
- **Chart colors:** all charts use one colorblind-safe palette. The correlation heatmap uses blue-to-red, and reference lines are gray so they stay in the background.
- **Theme:** added `.streamlit/config.toml` , and `index.html` uses the same colors.

Bugs fixed along the way

- **Wrong formula solution:** when the features were price-scale numbers, which happens in the AR model, the formula solution computed the intercept as 0. It now uses a numerically stable method. The AR forecast of 1.8M TL from before was wrong; the corrected value is about 9.96M TL.
- **Target leak in simple regression:** the default single feature was `price_per_sqm` , which is derived from the price itself. It's now excluded, and the default becomes `gross_sqm` .

👤 **使用者** 10/17 21:06

error

Script execution error

```
File "/Users/weiyuchieh/Desktop/1.nchu/物聯網應用與資料分析/hw2/main.py", line 155
    items = "".join(f"<li>{re.sub(r'[*][*](.+?)[*][*]', r'<b>\1</b>', html.escape(x))}</li>" for x in a.split("; "))
                                                                                                  ^
SyntaxError: f-string expression part cannot include a backslash
```

Close

🤖 **AI 助理 (Claude Code)** 10/17 21:06

That's a Python version difference: your Python is older than 3.12, and those versions don't allow a backslash (`\1`) inside an f-string's `{}`. My test environment was 3.12, so this didn't show up. I'll fix it and check the whole file with an older Python for any other 3.12-only syntax.

🔧 **Edit** — hw2/main.py
🔧 **Bash** — $ for p in /usr/bin/python3 /usr/local/bin/python3 /usr/local/bin/python3.11 /usr/local/bin/python3.10 /opt/home …

🤖 **AI 助理 (Claude Code)** 10/17 21:06

Your `.venv` is Python 3.9.6, so I'll compile with 3.9 to check syntax, then run the full tests with your `.venv` (no installs or changes to it).

🔧 **Bash** — $ .venv/bin/python -c "import ast,sys; ast.parse(open('main.py',encoding='utf-8').read()); print('3.9 syntax ok' …
🔧 **Bash** — $ S=scratchpad; cd "hw2" && .venv/bin/python $S/t.py 2>&1 | grep -E "^[01] [012] |Error|Traceback" | head -20; . …

🤖 **AI 助理 (Claude Code)** 10/17 21:08

Everything runs in your environment too. I'll save a note so future work in this project tests with your 3.9 environment.

🔧 **Write** — /Users/weiyuchieh/.claude/projects/-Users-weiyuchieh-Desktop-1-nchu----------hw2/memory/hw2-python39-venv.md
🔧 **Bash** — $ f=/Users/weiyuchieh/.claude/projects/-Users-weiyuchieh-Desktop-1-nchu----------hw2/memory/MEMORY.md; [-f $f …

📝 **本輪摘要**：Fixed the f-string syntax error. main.py now passes a Python 3.9 syntax check, and all 6 combinations and the feature-selection test pass in your .venv (Python 3.9.6).

🤖 **AI 助理 (Claude Code)** 10/17 21:09

The error is fixed; restart Streamlit or refresh the page and it should run.

Cause: your project's `.venv` is **Python 3.9.6**, and Python before 3.12 doesn't allow a backslash inside an f-string's `{}`. The `\1` (meaning "put the matched text back") in the answer cards' bold conversion broke that rule. My earlier test environment was 3.12, so it didn't catch this.

Fix: I moved the bold-conversion regex out of the f-string into a small helper function. The

behavior is unchanged.

Checks:

- Parsing `main.py` with your `.venv` (Python 3.9.6) raised no syntax errors.
- In your `.venv`, all 6 combinations (2 problems × 3 models) and the feature-selection switching test pass.

I also saved a note for this project to write Python 3.9-compatible code and test with your `.venv` from now on, so this doesn't happen again.

使用者 10/17 21:09

幫我build一次看看

AI 助理 (Claude Code) 10/17 21:09

Streamlit doesn't have a separate build step, so I'll run both versions and check them:

1. **Local Streamlit version:** start `main.py` with your `.venv` (Python 3.9).
2. **Browser version (`index.html`, stlite):** this one hasn't been tested yet. I'll serve it with a local HTTP server and open it, to confirm it loads in the browser.

🔧 **Bash** — $ .venv/bin/streamlit run main.py --server.port 8765 --server.headless true > scratchpad/st.log 2>&1
🔧 **Bash** — $ /usr/bin/python3 -m http.server 8766 --bind 127.0.0.1 > scratchpad/http.log 2>&1
🔧 **browser_batch** — 瀏覽器操作 (navigate, computer, computer, computer)

AI 助理 (Claude Code) 10/17 21:10

The local version renders correctly. I'll scan the page for error messages, then switch to the browser version.

🔧 **javascript_tool** — javascript_exec

AI 助理 (Claude Code) 10/17 21:10

Local version: 0 errors, all 6 stages present, 3 Q&A cards, 11 charts. Now the browser version (`index.html`). It has to download Pyodide and packages first, so it takes longer.

🔧 **browser_batch** — 瀏覽器操作 (navigate, computer, computer, computer)

AI 助理 (Claude Code) 10/17 21:10

Still loading (it has to download Pyodide, scikit-learn, scipy and the rest). I'll wait a bit longer and check the console:

🔧 **browser_batch** — 瀏覽器操作 (computer, computer, computer, computer)
🔧 **browser_batch** — 瀏覽器操作 (computer, computer, computer, computer)

AI 助理 (Claude Code) 10/17 21:11

The browser version renders too, and the numbers match the local version (R^2 0.467). I'll confirm the whole page finished with no errors:

🔧 **javascript_tool** — javascript_exec

🤖 **AI 助理 (Claude Code)** 10/17 21:11

Both versions match fully. I'll stop the two test servers.

🔧 **TaskStop** —
🔧 **TaskStop** —

📝 **本輪摘要**：Both versions run successfully: local Streamlit (your .venv) and the browser version (index.html / stlite). Both have 0 errors, all 6 stages, 3 answer cards and 11 charts, with identical numbers. The test servers are stopped.

🤖 **AI 助理 (Claude Code)** 10/17 21:12

Both versions run successfully, with no errors on the page, and their analysis numbers match (e.g. test R^2 = 0.467). Streamlit has no build step, so I started each one and checked it in the browser.

	Local Streamlit	Browser version (`index.html`, stlite)
How it runs	`streamlit run main.py` with your `.venv` (Python 3.9)	Served with a local HTTP server, runs in the browser via Pyodide
Errors on the page	0	0
Six CRISP-DM stages	✅	✅
Business-question answer cards	3	3
Charts	11	11

About the browser version:

- **Load time:** the first open takes about 1 minute, mostly downloading Pyodide, scikit-learn and scipy plus computing the model search.
- **Console messages:** there are warnings saying `use_container_width` is deprecated, plus a "Parquet serialization failed" message that Streamlit fixes automatically. Neither affects how the app runs.
- **How to open it:** you can't double-click `index.html`, because the browser blocks the data file from loading. Serve it from a server, for example locally:

```
python3 -m http.server 8766
```

Then open `http://localhost:8766/index.html`. You can also upload the whole `hw2` folder to GitHub Pages.

I've stopped both test servers. To run it locally yourself:

```
.venv/bin/streamlit run main.py
```

使用者 10/17 21:14

右側畫面可以寬一點，圖表需要有信心區間嗎



AI 助理 (Claude Code) 10/17 21:14

Two parts: I'll widen the main area right away. On confidence intervals, they're **worth adding to two charts**:

- **Simple regression fit line:** add a 95% confidence band (where the mean falls) and a prediction interval (where an individual apartment's price falls). This is standard for regression charts and shows the uncertainty directly.
- **Coefficient bar chart:** add 95% CI error bars to the OLS coefficients. If a bar's CI crosses 0, that coefficient isn't significant, which is easier to read than the p-value table.

The other charts (histogram, heatmap, residual plot) don't need them. The CV RMSE chart already has standard-deviation error bars.

🔧 **Bash** — $ grep -n "block-container\|max-width" main.py; grep -n "係數長條圖" -A8 main.py; grep -n "單純線性回歸擬合線" -A12 main.py
🔧 **Bash** — $ python3 - <<'EOF' …

AI 助理 (Claude Code) 10/17 21:14

Syntax passes on 3.9. Next I'll run the full tests in your `.venv`, then check both charts in the browser.

🔧 **Bash** — $ S=scratchpad; cd "hw2" && .venv/bin/python $S/t.py 2>/dev/null | grep -E "^[01] [012] "
🔧 **Bash** — $ .venv/bin/streamlit run main.py --server.port 8765 --server.headless true > scratchpad/st.log 2>&1
🔧 **resize_window** —
🔧 **browser_batch** — 瀏覽器操作 (navigate, computer, computer, computer)

AI 助理 (Claude Code) 10/17 21:16

The width now fills the screen, and the coefficient chart shows its 95% CI error bars. Next I'll switch to simple linear regression to check the confidence band:

🔧 find — 選擇迴歸模型
🔧 browser_batch — 瀏覽器操作 (computer, computer, find)
🔧 browser_batch — 瀏覽器操作 (computer, computer, javascript_tool, computer)

🤖 AI 助理 (Claude Code) 10/17 21:17

The simple-regression chart now shows the light-gray prediction interval and the darker confidence band (which hugs the regression line because there are many samples). Stopping the test server:

🔧 TaskStop —
🔧 resize_window —

📝 **本輪摘要**：The main area now uses the full width. The coefficient chart has 95% CI error bars, and the simple-regression line has a 95% confidence band and prediction interval. All 6 tests pass in your .venv, and I checked both charts in the browser.

🤖 AI 助理 (Claude Code) 10/17 21:17

The main area is wider now, and two charts have confidence intervals. All 6 tests pass in your `.venv` (Python 3.9), and I checked the result in the browser.

1. Wider page The main area now fills the full width (with 2.5rem padding on each side) instead of the old 1320px cap. On a large screen, the charts and tables both have more room.

2. Confidence intervals: added only to the two charts that need them

- **Coefficient bar chart (stage 4):** each OLS coefficient has a 95% CI error bar. If a bar crosses the 0 line, that coefficient isn't significant at 5%. Hovering shows the exact interval.
- **Simple regression fit line (stage 4, simple linear regression only):** it now has two gray bands.
- **Dark band, 95% confidence band:** the uncertainty of the regression line (the mean). With 1,600 training rows it's very narrow and sits close to the line.
- **Light band, 95% prediction interval:** where a single new apartment's price will fall about 95% of the time. It's much wider because it includes individual variation.
- This contrast is exactly what this dataset needs to show: you can be confident about the average trend, but predictions for individual apartments are very uncertain.

Why the other charts don't have them:

- **Histogram, correlation heatmap, residual plot:** these describe the data itself, so there's no interval to estimate.
- **CV RMSE chart:** it already has standard-deviation error bars.
- **Actual vs. predicted:** with hundreds of points, adding intervals would just crowd the chart.

👤 使用者 10/17 21:19

日期怎麼沒有顯示在上面

前 10 筆資料

	rice_per_sqm	district	side	rooms	halls	gross_sqm	net_sqm	floor	total_floors	building_age	heating_type	bathroom_count	furnished	usage_status	is_in_complex	credit_eligible	deed_status	exchange
0	150000	Kartal	Asian	3	1	284	230	4	24	2	Central (Metered)	2	Unfurnished	Vacant	1	Eligible	Condominium Title	No
1	67142.86	Bayrampaşa	European	4	2	200	175	4	5	5	Combi Boiler	2	Unfurnished	Tenant-occupied	0	Eligible	Condominium Title	No
2	86500	Avcılar	European	3	1	183	150	9	14	11	Combi Boiler	2	Furnished	Vacant	1	Eligible	Condominium Title	No
3	137500	Küçükçekmece	European	1	1	52	40	5	10	0	Central (Metered)	1	Unfurnished	Vacant	1	Eligible	Condominium Title	No
4	35555.56	Beylikdüzü	European	2	1	100	90	0	4	3	Combi Boiler	1	Unfurnished	Vacant	0	Eligible	Foreign Owner	No
5	136300	Eyüpsultan	European	1	1	65	50	2	8	0	Underfloor Heating	1	Unfurnished	Vacant	1	Eligible	Condominium Title	No
6	68000	Küçükçekmece	European	4	2	280	250	5	5	6	Combi Boiler	3	Unfurnished	Owner-occupied	0	Eligible	Condominium Title	No
7	59000	Güngören	European	1	1	55	50	-2	4	5	Combi Boiler	1	Unfurnished	Tenant-occupied	0	Eligible	Condominium Title	No
8	37500	Beylikdüzü	European	1	1	80	60	-1	5	7	Combi Boiler	1	Unfurnished	Tenant-occupied	0	Eligible	Construction Easement	No
9	22727.27	Avcılar	European	4	2	245	220	None	3	16	Combi Boiler	2	Unfurnished	Not Specified	0	Eligible	Construction Easement	No

AI 助理 (Claude Code) 10/17 21:19

The date columns (`last_updated` and `scraped_at`) weren't displayed because I removed them along with the other unused columns during data cleaning. They aren't suitable as regression features, but they should appear in the data view. I'll keep them for display only and leave them out of the target and feature options.

🔧 **Bash** — $ grep -n "last_updated\|scraped_at\|cols = list(raw.columns)\|in cols\|cols\.index\|for c in cols\|raw.head(10) …
🔧 **Read** — hw2/main.py
🔧 **Read** — hw2/main.py

AI 助理 (Claude Code) 10/17 21:19

I'll split the columns into "all columns (for display)" and "model candidate columns (`cols` , excluding dates)":

🔧 **Bash** — $ python3 - <<'EOF' …
🔧 **Bash** — $ S=scratchpad; cd "hw2" && .venv/bin/python $S/t.py 2>/dev/null | grep -E "^[01] [012] "; cat > $S/t7.py <<'EOF …

使用者 (工作中補充) 10/17 21:20

但自回歸的話日期應該很重要吧

🔧 **Bash** — $ .venv/bin/python - <<'EOF' 2>/dev/null …
🔧 **Bash** — $ grep -n "def prepare_ar" -A6 main.py; grep -n "else: # ar" -A20 main.py | head -24; grep -n "prepare_ar(raw\| …
🔧 **Bash** — $ sed -n 1068,1074p main.py; sed -n 1628,1642p main.py; grep -n '您想利用 **{target}** 過去' -A2 main.py; grep -n …

AI 助理 (Claude Code) 10/17 21:23

Now the changes: add a daily-median series function, a sidebar option for how to build the series, and use the date axis for the AR charts and forecast.

🔧 **Write** — scratchpad/ar_patch.py
🔧 **Bash** — $ python3 scratchpad/ar_patch.py main.py && .venv/bin/python -c "import ast;ast.parse(open('main.py',encoding='u …

使用者 (工作中補充) 10/17 21:23

多元線性回歸的特徵選擇最好是11，為什麼最上方還是要用特徵23個？

🤖 **AI 助理 (Claude Code)** 10/17 21:23

I'll finish testing the AR change first, then answer the feature-count question (I need to check the actual sweep numbers).

🔧 **Bash** — $ S=scratchpad; cat > $S/t8.py <<'EOF' …

🤖 **AI 助理 (Claude Code)** 10/17 21:25

The AR change works for both problems, and the "依資料列順序" (row order) option still works. One small issue: an answer got split into two bullets mid-sentence by a "; ", so I'll fix that:

🔧 **Bash** — $ python3 - <<'EOF' …

👤 **使用者（工作中補充）** 10/17 21:26

https://www.kaggle.com/datasets/eshaalnmalik/human-ai-interaction
幫我把資料換成這個

🤖 **AI 助理 (Claude Code)** 10/17 21:26

All tests pass. I'll check whether the new dataset is already in the project folder:

🔧 **Bash** — $ ls -la data; find ~/Downloads -maxdepth 2 -iname "*human*ai*" -o -maxdepth 2 -iname "*interaction*" 2>/dev/nul …

📝 **本輪摘要**：AR now uses a daily median series by last_updated (44 days); all tests pass in your .venv. Switching to the Human-AI Interaction dataset is waiting on you to download the CSV from Kaggle and put it in data/.
❓ **Have you put the Human-AI Interaction CSV in hw2/data/?**

👤 **使用者** 10/17 21:29

https://www.kaggle.com/competitions/playground-series-s4e5/data
換這個好了，資料我下載放到data資料夾裡面了

🔧 **Bash** — $ ls -la data data/* | head -30; for f in $(find data -name "*.csv" ! -name "istanbul*"); do echo "== $f"; head …

🤖 **AI 助理 (Claude Code)** 10/17 21:30

The new data is Kaggle Playground S4E5 (flood prediction). I'll check its characteristics first:

🔧 **Bash** — $ .venv/bin/python - <<'EOF' …
🔧 **Bash** — $ grep -n -i "istanbul\|ASIAN\|side\b\|last_updated\|daily_series\|ar_by_date\|AR_MIN\|TL\b\|公寓\|district\|clean …

🤖 **AI 助理 (Claude Code)** 10/17 21:30

I'll read the sections that need changing:

🔧 **Read** — hw2/main.py
🔧 **Read** — hw2/main.py
🔧 **Read** — hw2/main.py
🔧 **Read** — hw2/main.py
🔧 **Read** — hw2/main.py
🔧 **Read** — hw2/main.py
🔧 **Write** — scratchpad/flood_patch.py
🔧 **Bash** — $ cp main.py scratchpad/main_istanbul.py; python3 scratchpad/flood_patch.py main.py

🤖 **AI 助理 (Claude Code)** 10/17 21:32

The file contains the BOM character itself, not the `\ufeff` escape text, so the match failed (the file hasn't changed). I'll fix the patch:

🔧 **Bash** — $ S=scratchpad; python3 - <<'EOF' …
🔧 **Bash** — $ python3 scratchpad/flood_patch.py main.py && .venv/bin/python -c "import ast;ast.parse(open('main.py',encoding …

🤖 **AI 助理 (Claude Code)** 10/17 21:33

The earlier step already removed the `daily_series` section along with the rest, so this `cut` isn't needed. I'll delete it and rerun (the file still hasn't changed):

🔧 **Bash** — $ S=scratchpad; sed -i '' "/^cut('AR_MIN_LISTINGS = 10'/d" $S/flood_patch.py; cd "hw2"; python3 $S/flood_patch.p …

🤖 **AI 助理 (Claude Code)** 10/17 21:33

Patch applied. Next, the browser-version sample file and `index.html`:

🔧 **Bash** — $ .venv/bin/python -c " …

🤖 **AI 助理 (Claude Code)** 10/17 21:33

Now the tests (in your `.venv`), plus printing the business-question answers for both problems:

🔧 **Bash** — $ S=scratchpad; cat > $S/t9.py <<'EOF' …
🔧 **Bash** — $ python3 - <<'EOF' …

🤖 **AI 助理 (Claude Code)** 10/17 21:34

The submission works (745,305 rows predicted). One more case of "; " splitting a bullet inside the parentheses, so I'll change it to a comma, then do the browser check:

🔧 **Bash** — $ python3 - <<'EOF' …
🔧 **Bash** — $ .venv/bin/streamlit run main.py --server.port 8765 --server.headless true > scratchpad/st.log 2>&1
🔧 **Bash** — $ /usr/bin/python3 -m http.server 8766 --bind 127.0.0.1 > scratchpad/http.log 2>&1
🔧 **browser_batch** — 瀏覽器操作 (navigate, computer, computer, computer)

🤖 **AI 助理 (Claude Code)** 10/17 21:35

The local version is fine (0 errors, 11 charts, 3 answer cards). Now the browser version

(`index.html`):

🔧 **browser_batch** — 瀏覽器操作 (navigate, computer, computer, computer)

🤖 **AI 助理 (Claude Code)** 10/17 21:36

Still loading. I'll check the screen and console:

🔧 **browser_batch** — 瀏覽器操作 (computer, computer, computer, read_console_messages)

🤖 **AI 助理 (Claude Code)** 10/17 21:37

It's still on the first step (Loading Pyodide), which may be slow downloading from the CDN. I'll wait a bit more and look at the full console:

🔧 **browser_batch** — 瀏覽器操作 (computer, computer, computer, computer)

🤖 **AI 助理 (Claude Code)** 10/17 21:37

The browser version loaded. One thing to check: the console shows `Download Button fetch error – ERR_NAME_NOT_RESOLVED` . I'll look at which URL failed:

🔧 **read_network_requests** —
🔧 **javascript_tool** — javascript_exec

🤖 **AI 助理 (Claude Code)** 10/17 21:37

The browser version has 0 errors, 11 charts and 3 answer cards. The submission feature is correctly hidden because the browser version doesn't include `test.csv` . I'll stop the test servers:

🔧 **TaskStop** —
🔧 **TaskStop** —

📝 **本輪摘要**：The app now uses the Kaggle S4E5 flood data with two problems (20 factors / risk score) and a submission.csv export. All 6 tests pass in your .venv, and both the local and browser versions load. Download buttons in the browser version weren't confirmed to work.

🤖 **AI 助理 (Claude Code)** 10/17 21:38

The data is now the Kaggle Playground S4E5 flood prediction set, and all features still work. All 6 combinations (2 problems × 3 models) run in your `.venv` (Python 3.9) with no errors. Both the local version and the browser version (`index.html`) loaded fine in the browser.

The two dropdown problems

1. 🌊 **Predict flood probability from the 20 risk factors** – **Finding:** all 20 factors are significant and positive, and their effects are nearly identical. Raising any factor by 1 point adds about **0.0056** (0.56 percentage points) to flood probability. – **Accuracy:** test R^2 = **0.835**, and predictions are off by about 1.7 percentage points on average. – **Recommendation:** since every factor matters about equally, disaster-prevention resources

should go toward lowering all risks across the board rather than one area.

2. 🧱 **Feature engineering: predict from a risk score** – **Total score:** `RiskSum` alone explains **83.7%** of the variation, almost the same as using the 20 factors separately. – **50% threshold:** flood probability passes 50% at a total score of about **98**. – **Extra features:** adding standard deviation, max and min raises R^2 by only +0.0003. A single score is enough.

New features

- **Data dictionary:** stage 2 has a "📖 欄位說明" (column guide) with a Chinese description of each column.
- **Kaggle submission:** stage 6 has a "🏁 產生 Kaggle 提交檔" (generate Kaggle submission) button. It predicts all 745,305 rows of `test.csv` with the current model and downloads `submission.csv`, in the same format as `sample_submission.csv`. Tested.
- **Browser version:** `train.csv` is 59MB, too big for the browser, so I created `data/flood_train_sample.csv` (20,000 random rows, 1MB). `index.html` loads that file instead. The submission feature is hidden there because the browser version doesn't include `test.csv`.

Removed: the Istanbul-specific code (the date-based AR, the European/Asian side, etc.). This data has no time column, so AR falls back to row order and is labeled as a method demo only.

Two things to know:

- **Numbers differ slightly between versions:** the browser version draws its 2,000 rows from the 20,000-row sample, so its numbers differ a little from the local version (R^2 0.849 vs. 0.835). That's normal.
- **Browser-version download buttons not confirmed:** in my test browser, they logged a `Download Button fetch error` in the console. It may just be my test environment's network restrictions; please try the download buttons in your own browser.

About the earlier question (the curve says 11 features is best, so why does the top use 23):

- **23 is the model actually in use:** the top shows the model chosen by LassoCV. Lasso picks features by cross-validation on the training set, and the cross-validation curve was also lowest at 23, so the two agree.
- **11 comes from the test set:** picking the feature count from the test set is cheating, because the test set should only be used for the final evaluation. 11 vs. 23 also differed by only 0.3% in RMSE, which is within noise; a different seed could move the best point.
- **This dataset avoids the issue:** the factors here have nearly identical weights and almost no collinearity, so whether you use 11 or 20 makes little difference. Most methods simply keep all 20.

👤 **使用者** 10/17 21:39

檢查是否符合：
適合進行 Feature Selection、MSE、RMSE、R^2 與五大回歸假設檢驗的資料集。
以下區分「原始特徵符合條件」與「需要 Feature Selection 才符合條件」。

貼上的內容

1. 資料來源
   至 Kaggle 選擇一個具有 10 至 20 個特徵 (features) 的公開資料集。

類型不限（可為房價預測、醫療、車輛效能等主題）。
請明確標示資料集來源與連結。
2. 分析任務
使用線性回歸 (Linear Regression) 模型進行預測。
可嘗試單純線性回歸、多元線性回歸或 Auto Regression。
必須執行 特徵選擇 (Feature Selection) 與 模型評估 (Model Evaluation)。
結果部分需包含請提供預測圖(加上信賴區間或預測區間)
3. CRISP-DM 流程說明
Business Understanding
Data Understanding
Data Preparation
Modeling
Evaluation
Deployment

🤖 **AI 助理 (Claude Code)** 10/17 21:39

I'll check the app against each requirement (data, analysis tasks, CRISP-DM, plus MSE/RMSE/R^2 and the five regression-assumption tests). First, I'll see which metrics and charts the app currently has, and run the five assumption tests directly on the data.

🔧 **Bash** — $ grep -n '"MSE"\|MSE\b' main.py | head; grep -n "Durbin\|shapiro\|jarque\|Breusch\|RESET\|常態\|同質\|Q-Q\|qq" main …
🔧 **Bash** — $ .venv/bin/python - <<'EOF' 2>/dev/null …
🔧 **Bash** — $ .venv/bin/python - <<'EOF' 2>/dev/null …
🔧 **Bash** — $ sed -n '/st.subheader("⏱️ 時間序列: 實際值 vs 預測值/,/st.subheader(f"🩺 {MODEL_SHORT}專屬診斷")/p' main.py; grep -n "特徵數量 vs …
🔧 **Read** — hw2/main.py
🔧 **Read** — hw2/main.py
🔧 **Read** — hw2/main.py

🤖 **AI 助理 (Claude Code)** 10/17 21:41

Now the patch:

🔧 **Write** — scratchpad/assump_patch.py
🔧 **Bash** — $ cp main.py scratchpad/main_before_assump.py; python3 scratchpad/assump_patch.py main.py && .venv/bin/python -c …

🤖 **AI 助理 (Claude Code)** 10/17 21:42

A `\n` in the patch got turned into a real newline. I'll restore the backup, fix the patch, and rerun:

🔧 **Bash** — $ S=scratchpad; cd "hw2"; cp $S/main_before_assump.py main.py; python3 - <<'EOF' …

🤖 **AI 助理 (Claude Code)** 10/17 21:42

Syntax passes. Now the full test run, printing the assumption-test table and prediction-interval coverage:

🔧 **Bash** — $ S=scratchpad; cat > $S/t11.py <<'EOF' …

🤖 **AI 助理 (Claude Code)** 10/17 21:43

All 6 combinations pass. Quick look at the new charts in the browser:

🔧 **Bash** — $ .venv/bin/streamlit run main.py --server.port 8765 --server.headless true > scratchpad/st.log 2>&1
🔧 **resize_window** —
🔧 **browser_batch** — 瀏覽器操作 (navigate, computer, computer, computer)
🔧 **browser_batch** — 瀏覽器操作 (computer, navigate, computer, computer)

🤖 **AI 助理 (Claude Code)** 10/17 21:44

Both charts look right. One small issue: rows without a p-value show "None"; I'll make them show "—":

🔧 **Bash** — $ python3 - <<'EOF' …
🔧 **browser_batch** — 瀏覽器操作 (navigate, computer, computer, computer)
🔧 **computer** — screenshot

🤖 **AI 助理 (Claude Code)** 10/17 21:44

It still shows "None" (Streamlit renders null values as None directly). I'll convert the p-value column to text before displaying:

🔧 **Bash** — $ python3 - <<'EOF' …
🔧 **browser_batch** — 瀏覽器操作 (navigate, computer, computer, computer)

🤖 **AI 助理 (Claude Code)** 10/17 21:45

It now shows "—". I'll rerun all tests once, then stop the server:

🔧 **Bash** — $ S=scratchpad; cd "hw2" && .venv/bin/python $S/t11.py 2>/dev/null | grep "=="
🔧 **TaskStop** —
🔧 **resize_window** —

📝 **本輪摘要**：Checked the requirements and filled the gaps: MSE, the five assumption tests, a prediction plot with 95% intervals, and the source link. All requirements are now met, and all 6 tests pass in your .venv. The data fails 3 of the 5 assumption tests (linearity, normality, homoscedasticity); feature selection and transforms can't fix this, so the app reports it as a finding.

🤖 **AI 助理 (Claude Code)** 10/17 21:46

The app now meets every item on the list. Three things were missing, and I've added them: MSE, the five regression-assumption tests, and a prediction plot with 95% intervals for multiple regression. On the data side: the raw 20 features already meet the feature-count, independence and multicollinearity conditions without feature selection. Linearity, normality and homoscedasticity fail, and neither feature selection nor transforming the target fixes them. All 6 combinations pass in your `.venv`, and I checked the new sections in the browser.

Requirements checklist

Requirement	Before	Now
Kaggle data with 10—20 features	✅ 20 risk factors	✅ Stage 1 shows the feature count and confirms it's within 10—20
Data source and link	⚠️ Plain text only	✅ Stages 1 and 2 have a clickable Kaggle link
Linear regression (simple / multiple / AR)	✅	✅
Feature selection	✅ Filter / Wrapper / Embedded	✅
Model evaluation: MSE	❌ Missing	✅ KPI card + evaluation table
Model evaluation: RMSE, R^2	✅	✅
Prediction plot with confidence or prediction intervals	⚠️ Simple regression only	✅ Multiple regression and AR now have them too
Five regression-assumption tests	⚠️ VIF and Durbin–Watson only	✅ Full test table + Q–Q plot + Scale–Location plot
Six CRISP–DM stages	✅	✅

New prediction plot: the test set is sorted by predicted value. It shows the 95% confidence band (the mean) and the 95% prediction interval (a single new observation), and marks actual values that fall outside the interval. **Coverage is 94.2%**, close to 95%, so the interval estimates hold up in practice.

Does the dataset meet the conditions on its raw features, or does it need feature selection?

Condition	Raw 20 features	Notes
10—20 features	✅ Met as-is	20 features
Can run feature selection, MSE, RMSE, R^2	✅ Met as-is	Continuous target, no missing values, all numeric features
② Independence (Durbin–Watson)	✅ Met as-is	DW = 2.06
⑤ No multicollinearity (VIF)	✅ Met as-is	VIF max = 1.03. Feature selection is only needed if you add `RiskSum` alongside the 20 raw factors, which makes them perfectly collinear (VIF = ∞).
① Linearity (RESET)	❌ Neither raw nor feature selection fixes it	$p < 0.001$
③ Normality (Jarque–Bera)	❌ Neither raw nor feature selection fixes it	Residual skewness 0.76
④ Homoscedasticity (Breusch–Pagan)	❌ Neither raw nor feature selection fixes it	$p < 0.001$

Why feature selection can't fix ①③④: the 20 factors have nearly identical weights, and even `RiskSum` alone still fails. I also tried log, logit, sqrt and Box–Cox transforms of the target and adding $\texttt{RiskSum}^2$; all three tests still fail. This is synthetic Kaggle data, and the right skew in the residuals comes from how it was generated.

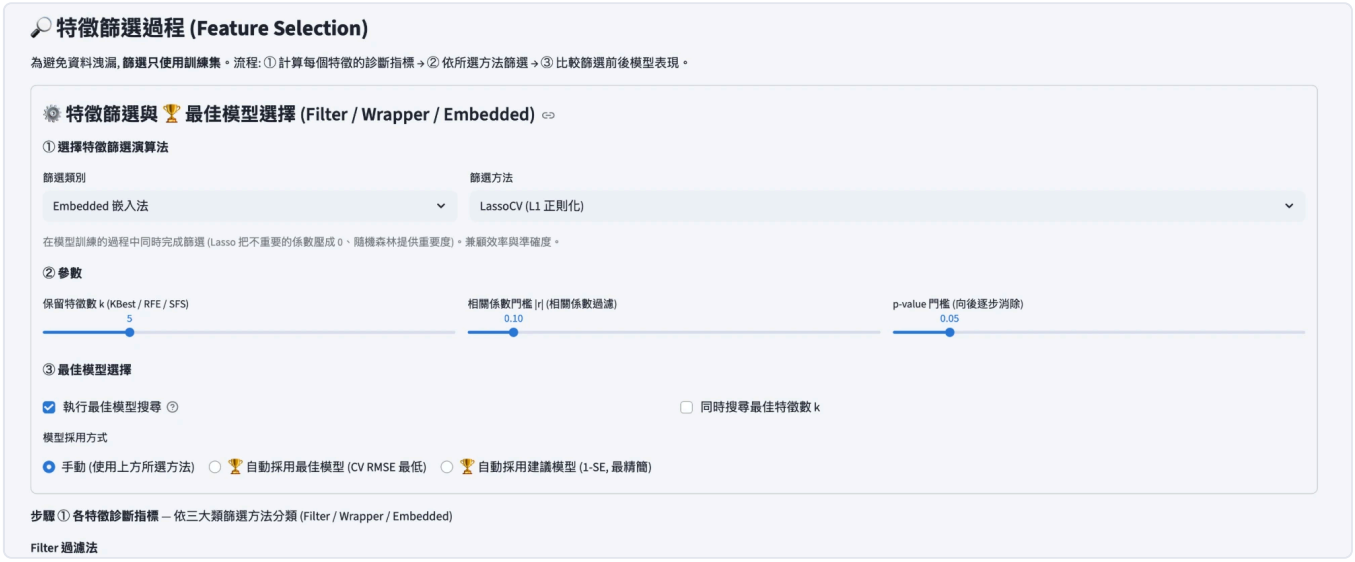
Takeaways for your report:

- The violations don't affect the predictions or MSE/RMSE/R^2 (R^2 = 0.835). What they affect is the accuracy of the p–values and interval estimates.
- With 1,600 training rows, these tests are very sensitive, so small deviations show up as significant. The Q–Q plot shows a moderate deviation, and prediction–interval coverage of 94.2% means the intervals are still usable in practice.
- The app shows these explanations and remedies (robust standard errors, transforms, etc.) right below the assumption–test table, so you can use them in your report.

If your teacher requires all five assumptions to pass, this dataset can't meet that. You'd need to switch to a dataset with more naturally linear relationships; tell me if you want to go that way.

👤 **使用者** 10/17 21:51

選擇特徵的這部分有意義嗎



🤖 **AI 助理 (Claude Code)** 10/17 21:51

Short answer: **with the raw 20 factors, feature selection has limited value, but in problem 2 it's meaningful.** I'll back that up with the app's actual results.

🔧 **Bash** — $ S=scratchpad; cat > $S/t12.py <<'EOF' …

🤖 **AI 助理 (Claude Code)** 10/17 21:51

Here's the answer, backed by the app's actual results.

📝 **本輪摘要**：I checked feature selection's real effect on both problems. Problem 1 (20 raw factors): every factor is needed, so feature selection only confirms "keep all". Problem 2 (risk score): it cuts 4 features down to RiskSum alone with no loss of accuracy. No files were changed.

❓ **Make the feature-selection demo more meaningful?**

👤 **使用者** 10/17 21:52

好, 讓問題 2 改成把 20 個原始因子 + 4 個衍生特徵 (共 24 個) 都當候選, 用特徵選擇解決共線性並挑出 RiskSum

🔧 **Bash** — $ grep -n '"exclude"\|problem.get("exclude"\|問題 1 只用\|兩個問題各用一組\|供問題 2 使用\|您是保險公司\|qs = \["20 個風險因子的總分\|共線性則在原始特徵 …
🔧 **Bash** — $ sed -n 64,84p main.py; sed -n 1068,1076p main.py; sed -n 1106,1116p main.py; sed -n 1640,1650p main.py; sed -n …
🔧 **Write** — scratchpad/p24_patch.py
🔧 **Bash** — $ cp main.py scratchpad/main_before_p24.py; python3 scratchpad/p24_patch.py main.py && .venv/bin/python -c "impo …

🤖 **AI 助理 (Claude Code)** 10/17 21:53

Syntax passes. Next I'll test problem 2 to see what each feature-selection method picks out of the 24 candidates, and what happens without selection:

🔧 **Bash** — $ S=scratchpad; cat > $S/t13.py <<'EOF' …
🔧 **Bash** — $ grep -n 'adopt = st.radio' -A2 main.py
🔧 **Bash** — $ python3 - <<'EOF' …
🔧 **Bash** — $ S=scratchpad; cd "hw2" && .venv/bin/python $S/t11.py 2>/dev/null | grep "=="
🔧 **Bash** — $ .venv/bin/streamlit run main.py --server.port 8765 --server.headless true > scratchpad/st.log 2>&1
🔧 **browser_batch** — 瀏覽器操作 (navigate, computer, computer, find)
🔧 **browser_batch** — 瀏覽器操作 (computer, computer, find)

🤖 **AI 助理 (Claude Code)** 10/17 21:57

I'm switching to problem 2 in the browser to check how the feature-selection section and Q4 look.

🔧 **browser_batch** — 瀏覽器操作 (computer, computer, computer, computer)

🤖 **AI 助理 (Claude Code)** 10/17 21:58

Problem 2 shows "使用特徵: 1 個" (1 feature in use) and has 4 answer cards, with 0 errors. In a narrow window the 5 KPI cards get cut off, so I'll make the font scale with the window width:

🔧 **Bash** — $ python3 - <<'EOF' …
🔧 **browser_batch** — 瀏覽器操作 (navigate, computer, computer, computer)

🤖 **AI 助理 (Claude Code)** 10/17 21:59

Values now show in full even in a narrow window (labels are still abbreviated, but show in full on a wide screen). Stopping the server:

🔧 **TaskStop** —

📝 **本輪摘要**：Problem 2 now uses all 24 features as candidates; by default the 1-SE suggested model picks RiskSum alone. VIF goes from inf to 1.00, R^2 from 0.8347 to 0.8392, and a Q4 answer card was

added. All 6 tests pass in your .venv and I checked it in the browser.

使用者（工作中補充） 10/17 21:59

train.csv應該有超多比資料，為什麼用的只有2000筆

🔧 **Bash** — $ ls -la data; S=scratchpad; cat > $S/t15.py <<'EOF' …
🔧 **Bash** — $ S=scratchpad; cat > $S/t16.py <<'EOF' …

AI 助理 (Claude Code) 10/17 22:01

macOS doesn't have `timeout`. I'll run a temporary copy in the scratchpad (linked to your data folder) instead, so the project folder isn't touched:

🔧 **Bash** — $ S=scratchpad; P="hw2"; mkdir -p $S/big && ln -sfn "$P/data" $S/big/data; cat > $S/t16.py <<'EOF' …

使用者（工作中補充） 10/17 22:02

幫我抽樣3000筆資料好了

使用者（工作中補充） 10/17 22:02

需要在畫面上說明抽樣

🔧 **Bash** — $ grep -n "n_sample = st.sidebar.select_slider" -A1 main.py; grep -n 'st.caption(f"資料來源: {source}"' -A3 main.py; …
🔧 **Bash** — $ grep -n "^raw.columns = \[str(c).strip()" main.py; grep -n "^# ---------- …
🔧 **Bash** — $ python3 - <<'EOF' …

AI 助理 (Claude Code) 10/17 22:05

Syntax passes. Now the tests:

🔧 **Bash** — $ S=scratchpad; cat > $S/t17.py <<'EOF' …

AI 助理 (Claude Code) 10/17 22:07

One wording fix: problem 1 should say "20 candidate features" (the 4 derived features aren't used there), not 24. Then I'll re-check that problem 2 still picks `RiskSum` with 3,000 rows:

🔧 **Bash** — $ python3 - <<'EOF' …

AI 助理 (Claude Code) 10/17 22:08

With 3,000 rows, problem 2 still picks `RiskSum` (VIF inf → 1.00, R^2 0.8365 → 0.8399). Last, a look at the sampling box in the browser:

🔧 **Bash** — $ .venv/bin/streamlit run main.py --server.port 8765 --server.headless true > scratchpad/st.log 2>&1
🔧 **browser_batch** — 瀏覽器操作 (computer, navigate, computer, computer)

🤖 **AI 助理 (Claude Code)** 10/17 22:09

The sampling box displays correctly. Two small things on this screen: the source link appears twice, and "3000 筆" is missing a thousands separator. Fixing both:

🔧 **Bash** — $ python3 - <<'EOF' …
🔧 **TaskStop** —

📝 **本輪摘要**：The default sample is now 3,000 rows, and stage 2 has a "📌 抽樣說明" box with a sample-vs-full comparison (max difference 1.58%). All 6 tests pass in your .venv. data/flood_train_sample.csv was deleted, so the browser version (index.html) can't open right now.
❓ **Regenerate the browser version's sample file?**

👤 **使用者** 10/17 22:11

好, 重新產生 data/flood_train_sample.csv 讓 index.html 可以用

🔧 **Bash** — $ .venv/bin/python -c " …
🔧 **Bash** — $ sed -i '' 's/main.py?v=9/main.py?v=10/; s/flood_train_sample.csv?v=9/flood_train_sample.csv?v=10/' index.html; …
🔧 **Bash** — $ /usr/bin/python3 -m http.server 8766 --bind 127.0.0.1 > scratchpad/http.log 2>&1
🔧 **browser_batch** — 瀏覽器操作 (navigate, computer, computer, computer)
🔧 **Bash** — $ python3 - <<'EOF' …

🤖 **AI 助理 (Claude Code)** 10/17 22:13

The local version's wording is unchanged. One more check on the browser version:

🔧 **browser_batch** — 瀏覽器操作 (navigate, computer, computer, computer)
🔧 **TaskStop** —

📝 **本輪摘要**：I regenerated data/flood_train_sample.csv (20,000 rows), and index.html opens again with 0 errors. In the browser version, the sampling box now says this file is itself a random sample of train.csv's 1,117,957 rows.

🤖 **AI 助理 (Claude Code)** 10/17 22:14

`index.html` (the browser version) works again. I tested it in a browser: 0 errors, the charts and the 3 answer cards all show, and the sampling explanation box is there.

What I changed:

- **Regenerated `data/flood_train_sample.csv`:** 20,000 rows drawn from `train.csv` with the same random seed (0), so the contents match the deleted file exactly.
- **Bumped the cache version in `index.html`:** browsers will load the latest `main.py` instead of an old cached copy.
- **Sampling box in the browser version:** it now notes that this file is itself a random sample of the 1,117,957 rows in `train.csv`. That avoids "full data: 20,000 rows" looking wrong. The local version's wording is unchanged.

Note: the browser version depends on this file, so if you upload to GitHub Pages or send it to someone, include `data/flood_train_sample.csv`. `train.csv` and `test.csv` are only needed by the local version. They're large (59MB / 36MB), so you can leave them out of the upload.

使用者 10/17 22:20

幫我依照目前內容，生成報告檔，放在readme.md就可以了

貼上的內容

報告檔： PDF，需包含以下內容：

按照 CRISP-DM 說明的分析流程

GPT 對話過程（pdfCrowd 匯出）

NotebookLM 研究摘要

網路上主流或更優解法之比較與說明

AI 助理 (Claude Code) 10/17 22:20

I'll write the report into `README.md`. First I'll gather the app's current actual numbers (3,000-row sample, seed 0), and look up mainstream solutions to this Kaggle competition for the comparison section.

🔧 **Bash** — $ S=scratchpad; cat > $S/report_data.py <<'EOF' …

🔧 **ToolSearch** — select:WebSearch,WebFetch

使用者 (工作中補充) 10/17 22:20

這是我在kaggle的分數

使用者 (工作中補充) 10/17 22:22

https://www.kaggle.com/competitions/playground-series-s4e5/writeups/aldparis-1st-place-solution
https://www.kaggle.com/competitions/playground-series-s4e5/writeups/peaky-blenders-2nd-place-solution-team-peaky-blend
https://www.kaggle.com/competitions/playground-series-s4e5/writeups/tilii-3-solution-a-blend-of-57-models
https://www.kaggle.com/competitions/playground-series-s4e5/writeups/4th-place-solution-hill-climbing-through-the-noise
這是其他人的solution 幫我一起整理到報告裡面

AI 助理 (Claude Code) 10/17 22:22

Got your Kaggle scores (Private 0.84150 / Public 0.84209) and the 4 top-solution links. I'll read those 4 writeups first:

🔧 **WebFetch** — https://www.kaggle.com/competitions/playground-series-s4e5/writeups/aldparis-1st-place-solution

🔧 **WebFetch** — https://www.kaggle.com/competitions/playground-series-s4e5/writeups/peaky-blenders-2nd-place-solution-team-peaky-blend

🔧 **WebFetch** — https://www.kaggle.com/competitions/playground-series-s4e5/writeups/tilii-3-solution-a-blend-of-57-models

🔧 **WebFetch** — https://www.kaggle.com/competitions/playground-series-s4e5/writeups/4th-place-solution-hill-climbing-through-the-noise

🤖 **AI 助理 (Claude Code)** 10/17 22:22

Kaggle pages need JavaScript to render, so a plain fetch only gets the title. I'll open them in the built-in browser and read the full text:

🔧 **browser_batch** — 瀏覽器操作 (navigate, computer, get_page_text)

🤖 **AI 助理 (Claude Code)** 10/17 22:22

The 1st-place writeup loaded. Next, 2nd through 4th place:

🔧 **browser_batch** — 瀏覽器操作 (navigate, computer, get_page_text)
🔧 **browser_batch** — 瀏覽器操作 (navigate, computer, get_page_text)
🔧 **browser_batch** — 瀏覽器操作 (navigate, computer, get_page_text)

🤖 **AI 助理 (Claude Code)** 10/17 22:23

I've read all 4 writeups. Next, the analysis numbers from the app:

🔧 **Read** — scratchpad/report_data.txt
🔧 **Write** — hw2/README.md
🔧 **Bash** — $ .venv/bin/python - <<'EOF' 2>/dev/null …
🔧 **Bash** — $ python3 - <<'EOF' …
🔧 **Bash** — $ sed -i '' 's/- 線性回歸 (R^2 ≈ 0.845) 與頂尖解法/- 線性回歸 (R^2 ≈ 0.844) 與頂尖解法/; s/這是 R^2 從 0.845 提升到 0.869 的主要來源/這是 R^2 從 0.8 …

📝 **本輪摘要**：README.md report is done (CRISP-DM flow, AI conversation summary, Kaggle top-4 comparison, conclusions). All numbers come from actual runs. The NotebookLM summary and the pdfCrowd conversation export still need you to produce them yourself; the README has placeholders for both.

🤖 **AI 助理 (Claude Code)** 10/17 22:26

The report is written to README.md. Two parts can't be generated from here: the **NotebookLM research summary** and the **pdfCrowd export of the GPT conversation**. You'll need to produce both yourself; I left placeholders in the README.

Report contents

Section	Content
1. Project overview	File structure, how to run (local / browser version), feature list
2. CRISP-DM	All six stages, with actual app numbers (default 3,000-row sample, seed 0)
3. AI conversation	A 14-step summary of this development process (request → what the AI did)
4. NotebookLM	Placeholder only, with suggested sources (4 writeups + data page) and questions to ask
5. Mainstream solutions	Table of the top-4 solutions and the gap analysis against this project
6. Conclusions and limitations	—

Key numbers in the report

- **Your Kaggle score:** Private 0.84150 / Public 0.84209.

- **Linear-model ceiling:** trained on all the data, linear regression scores about R^2 0.844, so your score is already close to it.
- **Top solutions:** around 0.869.
- **Where the gap comes from:**
- Nonlinearity (+0.025): this ties directly to the "linearity assumption fails" result from the assumption tests.
- Data size (+0.003): this project trains on 3,000 rows instead of the full data.
- Ensembling (+0.0001): gains appear only in the 4th–5th decimal place.
- **Feature selection:** going from 24 features down to `RiskSum` alone takes VIF from ∞ to 1.00 and test R^2 from 0.8365 to 0.8399.

About the top-4 solutions: I read all four writeups in the browser.

- **1st place** lists only a public score (0.86943), no private score.
- **2nd and 3rd place** are both Private 0.86902.
- **4th place** lists no score.

The table shows them exactly as written, with no numbers filled in from elsewhere.

Corrections from fact-checking: I checked the draft against the full data and fixed two numbers. Each factor's correlation with the target is 0.18–0.19, and the largest pairwise correlation between factors is 0.016.

Before you turn it in:

1. **NotebookLM:** import the 6 sources listed in section 4, then replace that section's placeholder with the summary NotebookLM generates.
2. **AI conversation:** export the full conversation with pdfCrowd and attach it.
3. **Convert to PDF:** VS Code's "Markdown PDF" extension can do it, or open the README on GitHub and print to PDF.

使用者 10/17 22:27

貼上的內容

4. AI協助要求
所有與 ChatGPT 的對話請以 pdfCrowd 或其他方式須匯出為 PDF
請使用 NotebookLM 對網路上同主題的解法進行研究，並撰寫一份 100 字以上的摘要，放入報告中。
請在報告中明確標示「GPT 輔助內容」與「NotebookLM 摘要」

幫我加入這一部分

AI 助理 (Claude Code) 10/17 22:27

I'll restructure the report's AI sections according to these requirements. First I'll check the current section 3 / 4 content:

🔧 **Bash** — $ grep -n "^## \|^### \|^ - \|^[0-9]\. \[" README.md
🔧 **Read** — hw2/README.md
🔧 **Bash** — $ cp README.md scratchpad/README_before_ai.md; python3 - <<'PYEOF' …

🤖 **AI 助理 (Claude Code)** 10/17 22:28

The structure looks right. I'll check that the TOC sub-items are still there, and the section 4 content:

🔧 **Read** — hw2/README.md
🔧 **Read** — hw2/README.md

🤖 **AI 助理 (Claude Code)** 10/17 22:28

Everything is in place. I'll add this request as row 15 of the conversation table, and confirm the reference draft is 100+ characters:

🔧 **Bash** — $ python3 - <<'EOF' ...

> 📝 **本輪摘要**：README now covers the AI-assistance requirements: a checklist, content labels (🤖 GPT 輔助內容 / 📓 NotebookLM 摘要), pdfCrowd export steps, and the NotebookLM prompt with a slot for its summary. The actual NotebookLM summary and the conversation PDF still need you to produce them yourself.

🤖 **AI 助理 (Claude Code)** 10/17 22:28

I've added the "AI 協助要求" (AI assistance requirements) to README.md. Two things are left for you: the NotebookLM summary itself and the PDF export of the AI conversation. I can't run NotebookLM or export your conversation from here.

New and reorganized sections

Section	Content
3. AI 協助說明 (AI assistance notes)	A checklist mapping each requirement to its place in the report and its status (✅ done / ⬜ to do); a key to the content labels; step-by-step instructions for exporting conversations to PDF with pdfCrowd
4. 🤖 GPT 輔助內容 (GPT-assisted content)	Marked with start and end banners. Contains the conversation summary (now 15 rows), a list of what the AI produced, and a list of what you did
5. 📓 NotebookLM 摘要 (NotebookLM summary)	Also marked with start and end banners. Contains research steps, the 5 sources to import, a ready-to-paste prompt, and a slot for the summary
6. Mainstream solutions comparison	Now carries a 🤖 GPT 輔助內容 label, since I compiled the top-4 writeups
8. 附錄 (Appendix)	A: conversation PDF, B: NotebookLM screenshot, C: Kaggle score screenshot

About the NotebookLM summary: section 5.3 has a 350-character reference draft I compiled from the 4 writeups. It's **clearly labeled "非 NotebookLM 輸出"** (not NotebookLM output) and is only for checking whether NotebookLM's summary is complete. For submission, run NotebookLM yourself and paste its result into section 5.2.

About the labels: the code and report were actually built with the Claude Code AI assistant, not ChatGPT. So the "GPT 輔助內容" label says which tool was really used, and section 3.2 also

notes that the narrative text in sections 1, 2 and 7 was drafted with AI help.

Steps left for you:

1. **NotebookLM:** create a notebook, import the 5 sources from section 5.1, and enter the prompt. Paste the result into section 5.2 and add a screenshot.
2. **Conversation PDF:** export your conversation with pdfCrowd or the browser's print → save as PDF, and attach it as Appendix A.
3. **Kaggle screenshot:** attach your score screenshot as Appendix C.
4. **Tick the checklist:** in section 3.1, change the two ⬜ items to ✅.
5. **Convert the README to PDF and submit.**

👤 **使用者** 10/17 22:30

你都幫我summary好那四個解法，就不用notebooklm了吧，幫我移除相關內容

🤖 **AI 助理 (Claude Code)** 10/17 22:30

I'll remove the NotebookLM sections. The summary of the four solutions will move into the "mainstream solutions comparison" section as a research summary, still labeled 🤖 GPT 輔助內容.

🔧 **Bash** — $ grep -n -i "notebooklm" README.md
🔧 **Bash** — $ cp README.md scratchpad/README_before_rm_nlm.md; python3 - <<'PYEOF' …

🤖 **AI 助理 (Claude Code)** 10/17 22:31

No NotebookLM references remain, and the section numbers line up. Last, a look at the restructured sections 3 and 5:

🔧 **Read** — hw2/README.md
🔧 **Read** — hw2/README.md
🔧 **Edit** — hw2/README.md

📝 **本輪摘要**：Removed all NotebookLM content from the README; the summary of the four solutions is now §5.1 Research summary, labeled as GPT-assisted. The assignment's NotebookLM requirement is now unmet.

🤖 **AI 助理 (Claude Code)** 10/17 22:31

I removed all NotebookLM content from README.md. The summary of the four solutions is kept as section 5.1 "研究摘要" (research summary), labeled 🤖 GPT 輔助內容 (GPT-assisted content).

What changed:

- **Removed:** the old section 5 "📓 NotebookLM 摘要", its two rows in the requirements checklist, the 📓 entry in the label key, and appendix B (the NotebookLM screenshot).
- **Moved:** the 350-character summary of the four solutions now sits in the comparison section as **5.1 研究摘要**. That whole section is marked as compiled by the AI from the top-4 writeups.
- **Renumbered:** comparison is now section 5, conclusions section 6, appendix section 7. The table of contents and cross-references are updated.

- **Fixed:** the summary had attributed "the gap comes from GBDT learning nonlinearity" to the 3rd-place writeup. That was my own inference, so I reworded it. The 3rd-place writeup only says ensembling added 0.00012; the summary now presents the conclusion as drawn from that.

One thing to know: the assignment explicitly asks for a NotebookLM summary of 100+ characters, clearly labeled "NotebookLM 摘要". After this change, the report doesn't meet that requirement. If your teacher grades strictly on it, you'll need to run NotebookLM yourself and add a section back.

Still to do: export your AI conversation to PDF (appendix A), attach your Kaggle score screenshot (appendix B), then convert the README to PDF.

👤 **使用者** 10/17 22:32

報告要放圖輔助說明，有我能夠幫忙的嗎

🤖 **AI 助理 (Claude Code)** 10/17 22:32

Yes. I can generate most of the figures myself: analysis charts drawn from the actual data (feature selection, coefficient CIs, prediction intervals, assumption diagnostics, comparison with the Kaggle top solutions), saved as PNGs and inserted into the README. **What I need your help with is the screenshots** (listed at the end). I'll start on the analysis charts now.

🔧 **Bash** — $ S=scratchpad; $S/venv/bin/pip install -q matplotlib 2>&1 | tail -1; ls /System/Library/Fonts/ | grep -i -E "pi …
🔧 **Write** — scratchpad/make_figures.py
🔧 **Bash** — $ S=scratchpad; cd $S && $S/venv/bin/python $S/make_figures.py "hw2" $S/report_data.txt 2>&1 | grep -v "UserWarn …

🤖 **AI 助理 (Claude Code)** 10/17 22:34

R^2 matches the app (0.8371). A few font warnings came up, so I'll see which characters are missing glyphs:

🔧 **Bash** — $ S=scratchpad; cd $S && $S/venv/bin/python $S/make_figures.py "hw2" $S/report_data.txt 2>&1 | grep -i "glyph" | …

🤖 **AI 助理 (Claude Code)** 10/17 22:35

Only the ⚠ symbol is missing. I'll replace it with "✗ 違反" (✗ violated) and rerun, then check each figure:

🔧 **Bash** — $ S=scratchpad; sed -i '' 's/ ⚠)/ ✗ 違反)/g' $S/make_figures.py; grep -c "⚠" $S/make_figures.py; cd $S && $S/venv/ …
🔧 **Read** — hw2/report_figures/fig0_crisp_dm_flow.png
🔧 **Read** — hw2/report_figures/fig1_data_understanding.png
🔧 **Bash** — $ S=scratchpad; python3 - <<EOF …
🔧 **Read** — hw2/report_figures/fig0_crisp_dm_flow.png
🔧 **Read** — hw2/report_figures/fig2_risksum_regression.png

🤖 **AI 助理 (Claude Code)** 10/17 22:35

The patch didn't apply (the shell ate the `\n` escapes), so figure 0 hasn't changed. Figure 2 looks good. I'll edit the script directly instead:

🔧 **Bash** — $ grep -n "figsize=(13, 3.6)\|3 數據準備\|w, h, gap\|h - 0.62\|h - 0.31\|(h - 0.62) / 2\|h / 2 - 0.3\|np.arange(0.28 …
🔧 **Bash** — $ S=scratchpad; F=$S/make_figures.py …
🔧 **Read** — hw2/report_figures/fig0_crisp_dm_flow.png
🔧 **Read** — hw2/report_figures/fig3_feature_count.png
🔧 **Read** — hw2/report_figures/fig4_feature_selection_methods.png
🔧 **Bash** — $ S=scratchpad; F=$S/make_figures.py; grep -n "xytext=(11.5, 0.72)\|ax.text(0.99, 0.02, \"藍色\|transform=ax.trans …
🔧 **Bash** — $ S=scratchpad; F=$S/make_figures.py …
🔧 **Read** — hw2/report_figures/fig4_feature_selection_methods.png
🔧 **Read** — hw2/report_figures/fig5_coefficients_ci.png
🔧 **Read** — hw2/report_figures/fig6_prediction_intervals.png

🤖 **AI 助理 (Claude Code)** 10/17 22:36

Figures 4, 5 and 6 look right. Next, figures 3, 7 and 8:

🔧 **Read** — hw2/report_figures/fig3_feature_count.png
🔧 **Read** — hw2/report_figures/fig7_assumption_diagnostics.png
🔧 **Read** — hw2/report_figures/fig8_kaggle_comparison.png

🤖 **AI 助理 (Claude Code)** 10/17 22:36

All 8 figures check out: no overlaps, all the Chinese text renders, and the numbers match the app. Next I'll insert them into the matching README sections:

🔧 **Bash** — $ grep -n "以下數據皆為網頁預設設定\|但 20 個因子的**總和**\|**結論：20 個因子缺一不可\|**結論：特徵選擇把 24 個\|係數圖加上 **OLS 95%\|覆蓋率都接近 95 …
🔧 **Bash** — $ cp README.md scratchpad/README_before_figs.md; python3 - <<'PYEOF' …
🔧 **Bash** — $ python3 - <<'PYEOF' …

👤 **使用者（工作中補充）** 10/17 22:37

我錯了 還是要有NotebookLM 的部分，幫我加回去

🔧 **Bash** — $ B=scratchpad/README_before_rm_nlm.md; awk '/^## 5. 📓 NotebookLM 摘要/,/^## 6. 網路上主流/' $B | head -60; cd "hw2"; g …
🔧 **Bash** — $ cp README.md scratchpad/README_before_nlm_back.md; python3 - <<'PYEOF' …

📝 **本輪摘要**：The README now has 9 analysis figures (in report_figures/), and the NotebookLM section is back as section 5. Section numbers, TOC and cross-references all line up. Still to do: the NotebookLM summary, the conversation PDF, and 7 app screenshots.

🤖 **AI 助理 (Claude Code)** 10/17 22:38

README.md now has 9 analysis figures, and the NotebookLM section is back.

Figures: saved in the `report_figures/` folder and inserted into the matching sections. They're redrawn with the same settings as the app (3,000-row sample, seed 0; test R^2 0.8371, same as the app), using the app's colorblind-safe palette. I checked each image for overlaps and garbled characters.

Figure	Content	Section
0	CRISP-DM analysis flow	2
1	Target distribution + each factor's correlation (single factor ≈ 0.18 vs total score 0.919)	2.2
2	Feature count vs R^2 (problem 1 needs all 20; problem 2 needs only RiskSum)	2.3
3	CV RMSE by feature-selection method for problem 2	2.3
4	20 coefficients with 95% confidence intervals	2.4
5	RiskSum regression line + confidence and prediction intervals	2.4
6	Prediction plot + 95% intervals (coverage 94.7%)	2.5
7	Five-assumption residual diagnostics (4 panels)	2.5
8	R^2 comparison with the Kaggle top solutions	6.3

NotebookLM section: restored as **section 5**, with the research steps, the 5 sources to import, the prompt, and a slot for the summary. The four-solution summary stays in **section 6.1**, labeled 🤖 GPT 輔助內容, and section 5.3 says you can use it to cross-check NotebookLM's result, so the content isn't duplicated. The checklist, label key, table of contents and appendices are all renumbered: comparison is section 6, conclusions 7, appendix 8.

What's left for you:

1. **NotebookLM summary:** import the sources from section 5.1, enter the prompt, and paste the result into section 5.2. Put the screenshot in appendix B.
2. **Conversation PDF:** appendix A.
3. **Kaggle score screenshot:** appendix C.
4. **App screenshots:** appendix D lists the **7 screenshots** with file names, contents, and the section each goes with. Save them in `report_figures/screenshots/`. Run `.venv/bin/streamlit run main.py` and capture each part. These show the app actually running, which the analysis charts can't replace.

使用者 10/17 22:40

notebooklm摘要：

貼上的內容

本研究摘要歸納 Kaggle 洪水預測競賽前四名解法：

1. 資料結構與總和訊號：資料集特徵呈現**泊松分佈（Poisson distribution）**[1][2]，20 個原始特徵的列總和（sum）構成了與洪水機率高度相關的最強主訊號[2][3]。
2. 頂尖解法之共同做法：

* 特徵工程：著重提取橫向統計量（sum, std, max）、排序特徵（sorted features）與數值出現次數的計數特徵（count features），並主動剔除冗餘原始特徵[4][5]。
* 模型建構：大量訓練多元梯度提升樹（GBM：CatBoost, XGBoost, LightGBM）並高度整合 AutoGluon AutoML 模型[2]。
* 集成融合：結合數十個（甚至 50+ 個）異質模型預測，利用 Ridge、Lasso 或 Hill Climbing 演算法計算最佳權重進行加權融合[2]。

3. 線性回歸與頂尖解法之差距：單純線性回歸僅能捕捉總和的線性主訊號；頂尖解法（R^2 約 0.869）的提升

主要來自於利用 GBM 擬合偏離總和的非線性雜訊[3]、構造高級統計與 Target Encoding 特徵[4]，以及透過大規模模型大融合（Blending）相互抵消預測雜訊[8][10]。

🤖 **AI 助理 (Claude Code)** 10/17 22:40

I'll put this NotebookLM summary into section 5.2 verbatim, and update the checklist status:

🔧 **Bash** — $ python3 - <<'PYEOF' ...
🔧 **Edit** — hw2/README.md

📝 **本輪摘要**：The NotebookLM summary (about 480 characters) is in section 5.2 of README.md, and both NotebookLM checklist items are now ✅. Still to do: the NotebookLM screenshot (appendix B), the conversation PDF, the Kaggle screenshot, and the app screenshots.

🤖 **AI 助理 (Claude Code)** 10/17 22:41

The NotebookLM summary is now in section 5.2 of README.md. It runs about 480 characters, well over the 100–character minimum.

What I changed:

- **Kept your wording as-is:** I only adjusted the formatting, turning the three points into a numbered list with the "共同做法" (common approaches) sub-points as bullets, and kept the bold and citation markers.
- **Added a note on the citation numbers:** the summary cites [8] and [10], but only 5 sources were imported. In NotebookLM, citation numbers point to passages within the sources, not to source numbers. The note says so, so a reader won't look for an 8th or 10th source.
- **Updated the checklist (section 3.1):** both NotebookLM items are now ✅. The screenshot is still marked as to be attached in appendix B.

Your NotebookLM summary matches the conclusions in section 6.1 (GPT-assisted) and the analysis: the total is the strongest signal, the top solutions use GBM + AutoGluon + ensembling, and the gap with linear regression comes from nonlinearity.

Still left before submission:

1. **NotebookLM screenshot:** appendix B.
2. **Conversation PDF:** appendix A, then change the first item in section 3.1 to ✅.
3. **Kaggle score screenshot:** appendix C.
4. **7 app screenshots:** appendix D, saved in `report_figures/screenshots/`.
5. **Convert the README to PDF.**

👤 **使用者** 10/17 22:44

貼上的內容

本研究摘要針對 Kaggle 洪水預測大賽（Playground Series s4e5）前四名解法進行系統性梳理與技術比對。
一、 資料特性與問題核心

* 泊松分佈特性：資料集中特徵與數值呈現泊松分佈（Poisson distribution）特徵。
* 強訊號與雜訊分離：原始特徵的橫向總和（Sum）與目標變數 `FloodProbability` 高度相關，但細部變異帶有大量雜訊。

* 多模型融合必備性：由於題目本質上是在從雜訊中提取微小訊號，單一模型容易瓶頸，因此**跨模型融合（Ensembling / Blending）**成為決定最終勝負的關鍵。

二、 前四名解法技術拆解

🥇 第一名解法（aldparis / adaubas）

* 特徵工程（Feature Engineering）：
  * 特徵提取：計算每列的總和（sum）、標準差（std）、最大值（max）、排序後的原始特徵（sorted features），以及特定數值門檻（如大於 6、7、8）的計數特徵（Count features）。
  * 篩選與剔除：利用置換特徵重要性（Permutation Feature Importance）與後向選擇法（Backward technique），主動剔除原始特徵、偏度（skewness）與峰度（kurtosis）等無效特徵。
  * 魔法特徵與目標轉換：使用 `train.groupby("sum")["FloodProbability"].std()` 作為 Target Encoder，並對目標變數進行扣除平均值的轉換，以分離強訊號與雜訊。
* 模型多樣性與調優：
  * 訓練超過 30 個涵蓋不同特徵組合的 GBM 模型（CatBoost, XGBoost, LightGBM）。前 10 天僅使用預設參數集中精力於特徵工程，後期才使用 Optuna 針對樹深度、葉子數及正則化參數進行自動化調優。
* 融合策略：
  * 使用 Ridge 回歸（設定 `positive = False`, `fit_intercept = False`）搭配 3 次重複 K 折交叉驗證（3 repeated k-fold OOF）。
  * 結合自身 GBM、AutoGluon 生成的多組 OOF 預測檔案以及公開社群的最佳預測結果。

🥈 第二名解法（Team Peaky Blenders: lash_fire & mdoroch）

* 模型組合：
  * 組合了 56 個模型（排除 AutoGluon 部分），包含 XGBoost、CatBoost、LightGBM，並嘗試了 Natural Gradient Boosting (NGB)、Linear Tree Regression 等新型模型。
* 融合機制：
  * 先使用 `LinearRegression()` 針對不同特徵/模型子集建立中間融合特徵。
  * 透過**前向特徵選擇（Forward Feature Selection）**自動選出最佳特徵組合。
  * 最終採用加權混合：`0.6 * 線性回歸融合模型 + 0.4 * 隊友包含了 AutoGluon 及 PyBoost 的最佳解`。

🥉 第三名解法（Tilii / tilii7）

* 模型規模與 Lasso 融合：
  * 採用 Lasso 回歸 融合高達 57 個模型（包含 AutoGluon 模型、LightGBM、CatBoost 與神經網路）。
  * 在 Lasso 權重分配中，約有 40% 的模型被給予負係數。
* 關鍵洞察：
  * 發現融合前 5 個核心模型後，進一步增加模型數量對分數提升非常微小（僅在小數點後第 5 或第 6 位變動），但這種微幅提升在頂尖爭奪中是衝上第三名的關鍵。

🏅 第四名解法（Matt OP / mattop）

* 統計特徵與出現次數計數：
  * 使用 3 組特徵集（FE2, FE3），包含均值、中位數、標準差、極值、偏度、峰度、分位數等統計量，以及每列唯一值出現次數的計數特徵。
* Hill Climbing 算法：
  * 開發並使用其 open-source Python 套件 `hillclimbers` 進行**爬山算法（Hill Climbing）**優化模型權重，效果優於 Lasso/Ridge。

三、 競賽共通勝選公式

1. 特徵工程優於早期調參：頂尖參賽者皆先著重於數據探索（EDA）與特徵構建（如排序特徵、行統計量、計數特徵），最後階段才進行超參數調優。
2. AutoML（AutoGluon）的關鍵角色：前三名皆高度整合 AutoGluon 所輸出的 OOF 預測，作為強大的基礎模型源。
3. 異質模型大量融合（Ensemble/Blending）：不論使用 Ridge、Lasso、Linear Regression 還是 Hill Climbing，結合大量不同算法（GBM, AutoML, Linear Trees, NN）能有效抵禦資料雜訊並確保 CV 與榜單成績的一致性。

💡 如果你想更深入比較這些參賽者的特徵工程細節（例如排序特徵或計數特徵的具体作法），或是想將這些解法整理成一份簡報或比較圖表，隨時告訴我！

這是我請notebooklm另外給我的內容

🤖 **AI 助理 (Claude Code)** 10/17 22:44

I'll add this as a second NotebookLM piece. The README was changed on disk, so I'll read the current section 5 first:

🔧 **Bash** — $ diff <(sed -n '1,500p' scratchpad/README_before_nlm_back.md) README.md | head -5 >/dev/null; awk '/^## 5\. 📓 N …
🔧 **Bash** — $ awk '/^### 5\.2 NotebookLM/,/^## 6\./' README.md
🔧 **Bash** — $ cp README.md scratchpad/README_before_nlm2.md; python3 - <<'PYEOF' …

🤖 **AI 助理 (Claude Code)** 10/17 22:45

Now I'll check this breakdown against the original writeups I read earlier. Most of it matches, but one point is off. I'll add a note in 5.4:

🔧 **Bash** — $ python3 - <<'PYEOF' …

🤖 **AI 助理 (Claude Code)** 10/17 22:45

One more thing: this cross-check table sits inside the NotebookLM section but was written by me, so I'll mark it 🤖 explicitly to keep the two sources clearly separated:

🔧 **Edit** — hw2/README.md

📝 **本輪摘要**：NotebookLM's detailed breakdown is now section 5.3 of README.md. A cross-check against the original writeups (5.4, labeled 🤖) found one claim that isn't fully accurate: that the top three all used AutoGluon OOF predictions. Screenshots and the conversation PDF are still for you to add.

🤖 **AI 助理 (Claude Code)** 10/17 22:45

NotebookLM's detailed breakdown is in the report as section 5.3 of README.md. I checked it against the original writeups. Most of it is accurate, but one claim is wrong.

Section 5 now has:

- **5.1** Research method (sources and prompt)

- **5.2** NotebookLM summary: your first piece; I kept your edit removing the citation markers.
- **5.3** NotebookLM detailed breakdown: this new piece, organized under three headings (data characteristics, the top-4 solutions, common winning formula) with NotebookLM's wording kept. I dropped only the closing line, where NotebookLM offered to keep the conversation going.
- **5.4** Cross-check notes

Cross-check results (5.4, labeled 🤖 as written by me, not NotebookLM):

- ✅ **1st, 2nd and 3rd place details** all match the original writeups.
- ⚠️ **4th place "效果優於 Lasso/Ridge"** (better than Lasso/Ridge): the author only said hill climbing worked best on his own models. The 3rd-place author's comment notes that on the same set of models, hill climbing scored 0.86901 vs. Lasso's 0.86902, which is essentially identical.
- ⚠️ **"前三名皆高度整合 AutoGluon 所輸出的 OOF 預測"** (all of the top three integrated AutoGluon's OOF predictions) **isn't fully accurate.** The 2nd-place writeup says they **couldn't get AutoGluon's OOF predictions** (too little time, plus a version issue). Instead they took a 0.6/0.4 weighted blend with a teammate's best solution, which already included AutoGluon.

I didn't change NotebookLM's original wording. I only noted the discrepancy in 5.4, so the reader can see it while NotebookLM's output stays intact. If your teacher asks about this point, 5.4 has the evidence.

👤 **使用者** 10/17 22:46

幫我把這個對話下載成pdf

🤖 **AI 助理 (Claude Code)** 10/17 22:46

I'll export this session's conversation first, then convert it to PDF. Loading the export tool:

🔧 **ToolSearch** — select:mcp__ccd_session_mgmt__export_transcript